

Adversarial Search Algorithms in Artificial Intelligence (AI)

Last Updated : 23 Jul, 2025



Adversarial search algorithms are the backbone of strategic decision-making in artificial intelligence, it enables the agents to navigate competitive scenarios effectively. This article offers concise yet comprehensive advantages of these algorithms from their foundational principles to practical applications. Let's uncover the strategies that drive intelligent gameplay in adversarial environments.

Table of Content

- [What is an Adversarial search?](#)
- [Adversarial search algorithms](#)
- [Minimax algorithm](#)
- [Alpha-beta pruning](#)
- [Adversarial search algorithm Implementations using Connect-4 Game](#)
- [Applications of adversarial search algorithms](#)
- [Conclusion](#)
- [FAQ's on Adversarial search algorithms](#)

What is an Adversarial search?

The Adversarial search is a well-suited approach in a competitive [environment](#), where two or more [agents](#) have conflicting goals. The adversarial search can be employed in two-player [zero-sum games](#) which means what is good for one player will be the misfortune for the other. In such a case, there is **no win-win outcome**. In [artificial intelligence](#), adversarial search plays a vital role in decision-making, particularly in competitive environments associated with games and strategic interactions. By employing adversarial search, AI agents can make optimal decisions while anticipating the actions of an opponent with their opposing objectives. It aims to establish an effective decision for a player by considering the possible moves and the counter-moves of the opponents.

The adversarial search in competitive environments can be utilized in the below scenarios where the AI system can assist in determining the best course of action by both considering the possible moves and counter-moves of the opponents.

- Each agent seeks to boost their utility or minimize their loss.
- One agent's action impacts the outcomes and objectives of the other agents.
- Additionally, strategic uncertainty arises when the agents may lack sufficient information about each other's strategies.

Role of Adversarial Search in AI

- **Game-playing:** The Adversarial search finds a significant application in [game-playing](#) scenarios, including renowned games like chess, Go, and poker. The adversarial search offers the simplified

nature of these games that represents the state of a game in a straightforward approach and the agents are limited to a small number of actions whose effects are governed by precise rules.

- **Decision-making:** Decision-making plays a central role in adversarial search algorithms, where the goal is to find the best possible move or strategy for a player in a competitive environment against one or more components. This requires strategic thinking, evaluation of potential outcomes, and adaptive decision-making throughout the game.

Adversarial search algorithms

The search algorithms like DFS, BFS, and A* can be well-suited for single-agent environments where there is no direct competition or conflict between multiple agents. These algorithms are suitable for finding an optimal solution in such scenarios. On the other hand, in **zero-sum games** where two players compete directly against each other, **adversarial search algorithms** like **Minimax** and **Alpha-Beta pruning** are more appropriate since these algorithms can determine the best course of action for each player in zero-sum games.

Minimax algorithm

The Minimax algorithm is claimed to be a recursive or backtracking algorithm that is responsible for choosing the best optimal move in the conflicting environment. The Minimax algorithm operates on a tree structure known as the game tree, which is the collection of all the possible moves in the corresponding game states in a given game. The game tree's leaf node accommodates all the possible moves. The game state denotes the current board condition. With every single move, the game state changes and the game tree gets updated height-wise. When visualized, this game tree often resembles an inverted tree, with the root representing the current game state and the branches representing possible moves.

This algorithm simply works by proceeding down to the tree until it reaches the leaf node and then it backs up the minimax values through the tree as the recursion unwinds. The prime motive of the Minimax algorithm is to maximize the chance of winning for the maximizer. In the game tree, every node is assigned weights that influence the chances of winning, the higher the weights, the higher the chances of winning.

Key terminologies in the Minimax algorithm

- **Minimax Tree:** A tree structure shows all possible moves and game states in a game, used by Minimax to find the best move.
- **MAX (Maximizer)** - Maximizer seeks favorable outcomes in games by maximizing chances of winning for themselves as players.
- **MIN (Minimizer)** - Minimizer reduces winning chances, making moves leading to least favorable outcome for the Maximizer in games.
- **Initial state** - Starting state of game board, representing the configuration at the beginning of the game.

- **Terminal state** -Terminal states are the final outcomes of a game board, resulting in either a win, loss, or draw.
- **Heuristic Evaluation Function:** Function evaluates game states for maximizer, assigning numerical values to each game state based on piece positions, material advantage, and board control.

How Minimax search algorithm help in finding the optimal strategy?

- **Terminal state:** The Minimax value of each state in the game tree can be used to determine the optimal strategy. The MINIMAX(s) refers to the utility of being in that state that assumes both players play optimally from there to the end of the game.
- **Non-terminal state:** In the case of the non-terminal state, when MAX's turn arrives at the game, MAX aims to move which leads to a state of maximum value, conversely MIN prefers to choose the move that leads to a state of minimum value. It can be formulated as below:

$$MINIMAX(S) = \begin{cases} UTILITY(s, MAX) \rightarrow \text{if } IS_TERMINAL(s), \\ \max_{a \in Actions(s)} MINIMAX(RESULT(s, a)) \rightarrow \text{if } TO_MOVE(s, MAX), \\ \min_{a \in Actions(s)} MINIMAX(RESULT(s, a)) \rightarrow \text{if } TO_MOVE(s, MIN) \end{cases}$$

Pseudocode for Minimax algorithm

The Pseudocode for the Minimax algorithm is given below:

- **MINIMAX(node, depth, maximizingPlayer):** This function recursively evaluates the minimax value of each node in the game tree. It alternated between maximizing and minimizing player nodes based on the maximizingPlayer parameter

```
MINIMAX(node, depth, maximizingPlayer):
    if depth is 0 or node is a terminal node:
        return the heuristic value of node
    if maximizingPlayer:
        bestValue = -INFINITY
        for each child of node:
            value = MINIMAX(child, depth - 1, FALSE)
            bestValue = max(bestValue, value)
        return bestValue
    else:
        bestValue = +INFINITY
        for each child of node:
            value = MINIMAX(child, depth - 1, TRUE)
            bestValue = min(bestValue, value)
        return bestValue
```

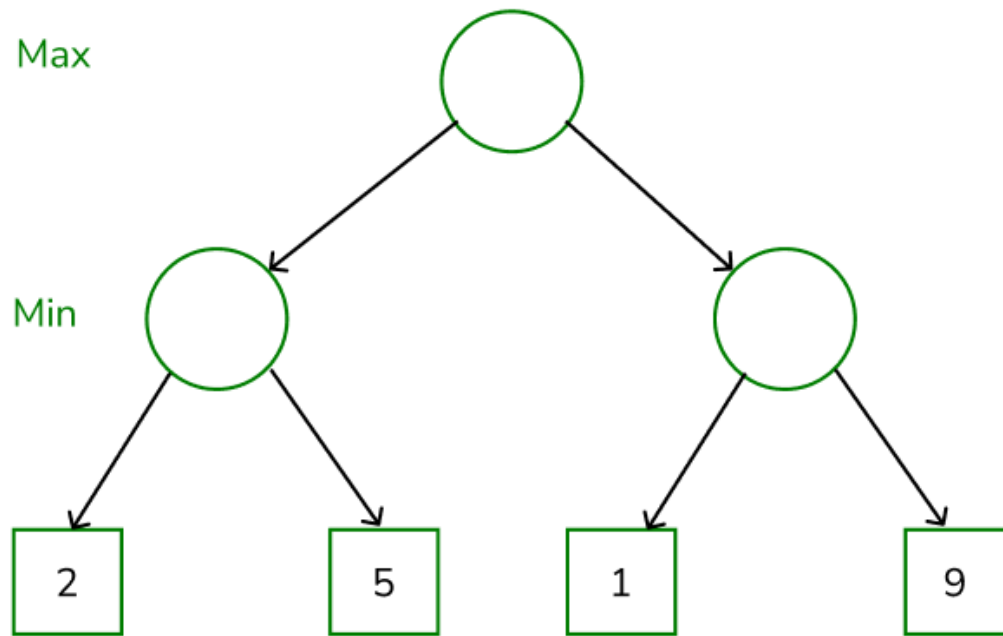
- **findBestMove(board):** This function is responsible for iterating through all possible moves on the board and evaluates each moves using minimax algorithm. Then it returns the moves with the **highest minimax value for the maximizing player**.

```
findBestMove(board):  
    bestValue = -INFINITY  
    bestMove = NULL  
    for each possible move in board:  
        if move is valid:  
            make the move on the board  
            value = MINIMAX(board, depth, FALSE)  
            undo the move on the board  
            if value > bestValue:  
                bestValue = value  
                bestMove = move  
    return bestMove
```

Understanding the Minimax algorithm: Decision-making in game trees

The minimax algorithm is a **backtracking algorithm** that **recursively** tries different possibilities and backtracks when the solution is not found. In the context of the Minimax algorithm, this involves exploring all possible moves in the game tree that evaluate each potential outcome and making decisions based on the evaluations.

The diagram below represents the [binary tree](#) that holds the possible game states and moves in a game, with each leaf node representing a final game state.



figure(a) Example of Minimax algorithm that holds the possible game states

Maximizer and Minimizer

In the Minimax algorithm, there are **two players such as the maximizer and the minimizer**. The maximizer aims to maximize its score (e.g., by winning the game), conversely, the minimizer aims to minimize the maximizer's score (e.g., by preventing the maximizer from winning).

Example of decision-making

- **Initial decision-making:** An example scenario in Figure (a) within the perfect binary tree causes the maximizer to make the first move at the root. The maximizer has the option to choose to traverse through the tree, in this case, it needs to choose the path between left or right.
- **Minimizer's choice:** Assuming that maximizer traverses to the left where it reaches the minimizer node, the minimizer has the responsibility of selecting the optimal value and it chooses between two values (2 and 5). The objective of the minimizer is to minimize the maximizer's score, so it chooses the lowest value 2. The value 2 is then passed to the root node so that it can be compared later when the player earns an optimal value when he traverses to the right side of the game tree.

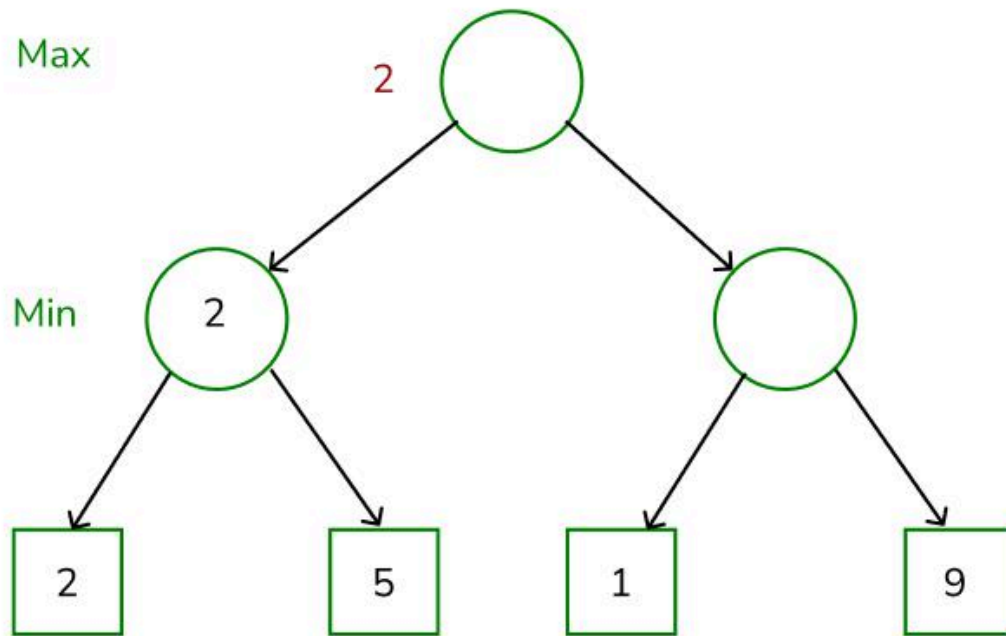


Figure (b) Minimizer's choice

- **Traversal to the Right:** The maximizer then traverses to the right side of the game tree and encounters another set of decision nodes. Eventually, it reaches another minimizer node where the minimizer again selects the optimal minimal value 1 among its options.

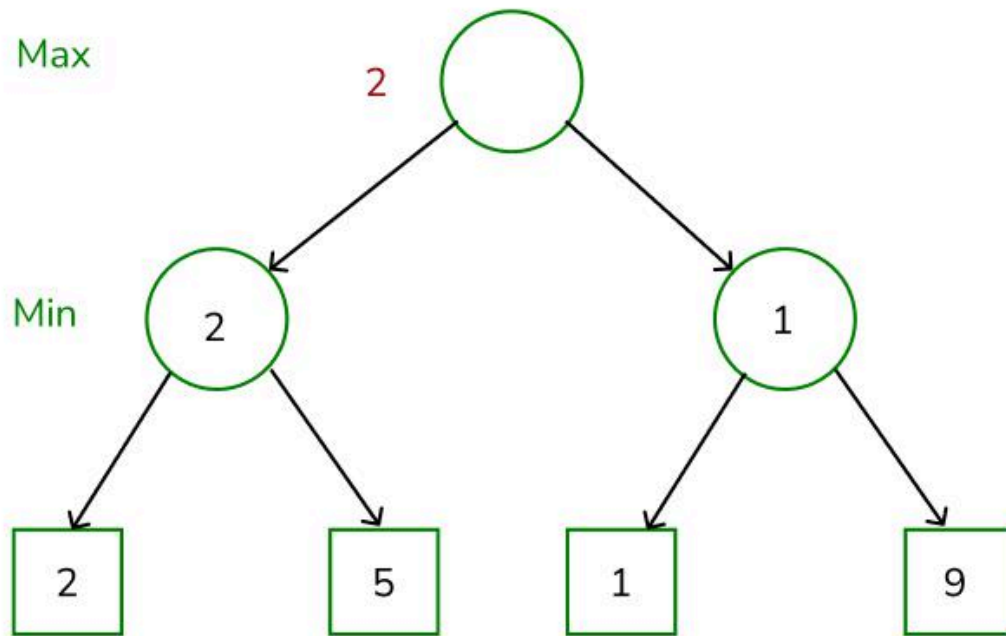
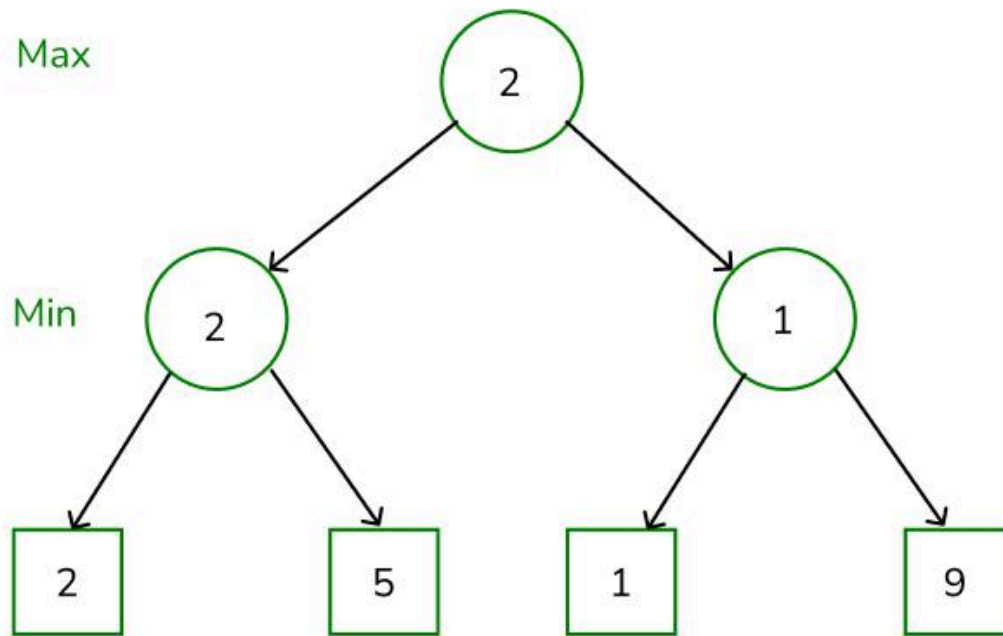


Figure (c) The maximizer traversing through the right

- **Recursive process:** The process continues recursively where each player makes decisions based on whether they are maximizer or minimizer and the objective of maximizing or minimizing the score respectively.
- **Comparison and outcome:** The maximizer node (root node) already holds the value of 2, when the player earns another optimal value that is 1 while traversing in the right path, it is then compared to get the final optimal value. The maximizer earns the optimal value 2 based on the decisions made by both players.



Final outcome of binary game tree

Time and Space complexity of Minimax algorithm

Time complexity

The minimax algorithm explores the complete game tree using a depth-first search approach that considers all possible moves at each level. The [time complexity](#) of the minimax algorithm can be written as,

$$O(b^m)$$

b - denotes the legal moves available at each point,

m - denotes the maximum depth of the tree.

This complexity arises because, at each level of the tree, the minimax algorithm must consider all *b* legal moves and this process repeats recursively for *m* levels until a terminal state is reached.

Space complexity

The [space complexity](#) of the minimax algorithm relies on how the algorithm generates and stores the actions. In the case, where if all the possible actions are generated and stored simultaneously, the space complexity can be written as

$$O(bm)$$

b - denotes the legal moves available at each point,
m - denotes the maximum depth of the tree.

This arises because the algorithm needs to store all the possible actions at each level of the game tree which leads to exponential growth in space requirements with the depth of the tree.

However, if the actions are generated and processed one at a time, the space complexity reduces to $O(m)$.

Alpha-beta pruning

[Alpha-beta pruning](#) is an optimization technique for a minimax algorithm. It reduces computation time by a huge factor, allowing the user to traverse faster and deeper into the tree. It stops evaluating when at least one possibility has been found that typically proves the move to be worse than the previously examined move. The minimax search is based on depth-first search which considers the nodes along a single path in a tree. But Alpha-Beta pruning bonds with two major parameters in MAX-VALUE(state, alpha, beta), representing Alpha plays a maximizer role, whereas Beta plays a minimizer role.

- **Alpha** - denotes the value of the best or highest value.
- **Beta** - denotes the value of the best or lowest value.

Why Alpha-beta pruning is considered as an effective algorithm in the adversarial search?

- **Branch elimination:** The Alpha-beta pruning algorithm relies on the branches of the game tree that can be eliminated which promotes the more promising subtree by limiting the search time.
- **Branch and bound:** Alpha-beta pruning is based on the [branch and bound](#) method which offers a way to solve the optimization problem by dividing them into sub-problems. It allows the algorithm to effectively remove the branches that did not yield the optimal solution.
- **Heuristic improvements:** To limit the computation time, the heuristic Alpha-beta tree search cuts off the search early and applies the heuristic evaluation function to states that treat the non-terminal nodes as if they were terminal nodes.

Pseudocode for Alpha-beta pruning

The Pseudocode for Alpha-beta pruning is given below

```
function ALPHA-BETA(node, depth, alpha, beta, maximizingPlayer):  
    if depth == 0 or node is a terminal node:  
        return the heuristic value of node
```

```

if maximizingPlayer:
    for each child of node:
        alpha = max(alpha, ALPHA-BETA(child, depth - 1, alpha, beta, FALSE))
        if beta <= alpha:
            break    // Beta cut-off
    return alpha
else:
    for each child of node:
        beta = min(beta, ALPHA-BETA(child, depth - 1, alpha, beta, TRUE))
        if beta <= alpha:
            break    // Alpha cut-off
    return beta

```

Understanding the Alpha-beta pruning: Decision-making in game trees

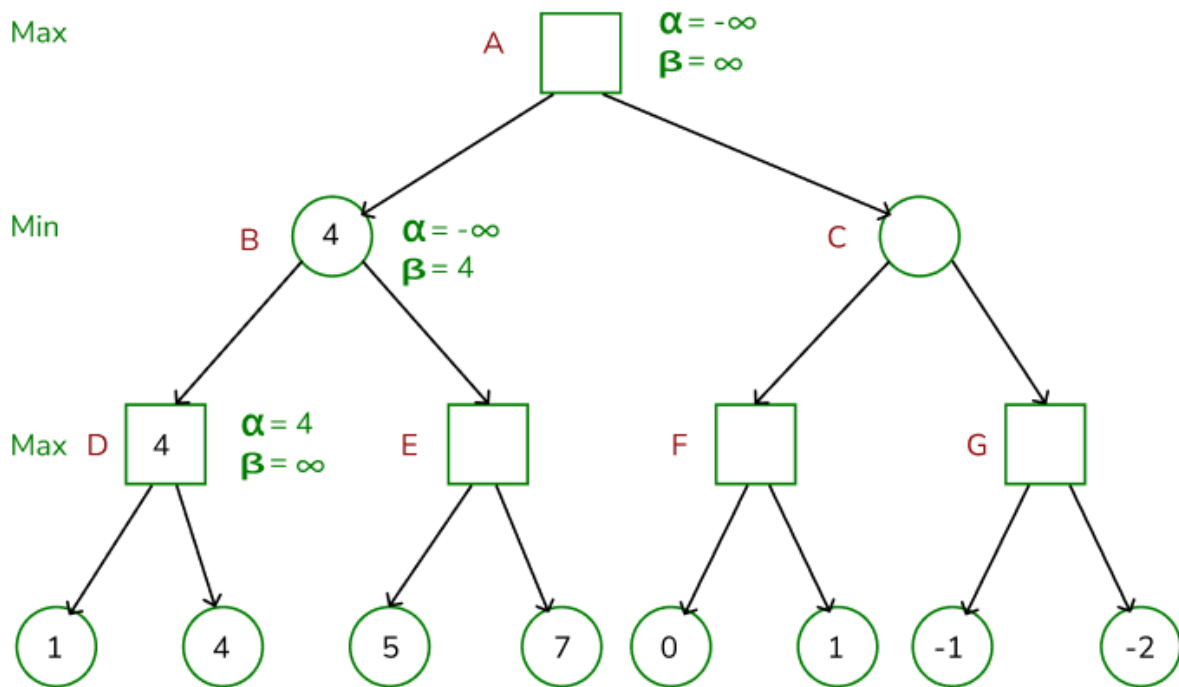
Game trees can hold abounding branches that can lead to different game states and implementing alpha-beta pruning in such game trees can help to prune away the branches if there already exists a better move. This makes the algorithm computationally efficient since there is no need to search the other branches if there already exists a better move in the game tree.

At any node in the game tree, the maximizing player is used to store the maximum score in the alpha which assures that the max node contains only the maximum scores. Similarly, the minimizing player is assured of a minimum value which is stored by the beta. These two values of alpha and beta are updated and maintained by the algorithm.

Take a look at the below example, initially both players begin with the worst possible values they can score, therefore the alpha is assigned with the value of minus infinity, whereas the beta is assigned with the value of plus infinity. We have a condition to prune the branches, if the alpha is greater than or equal to the beta value then we can prune the branch.

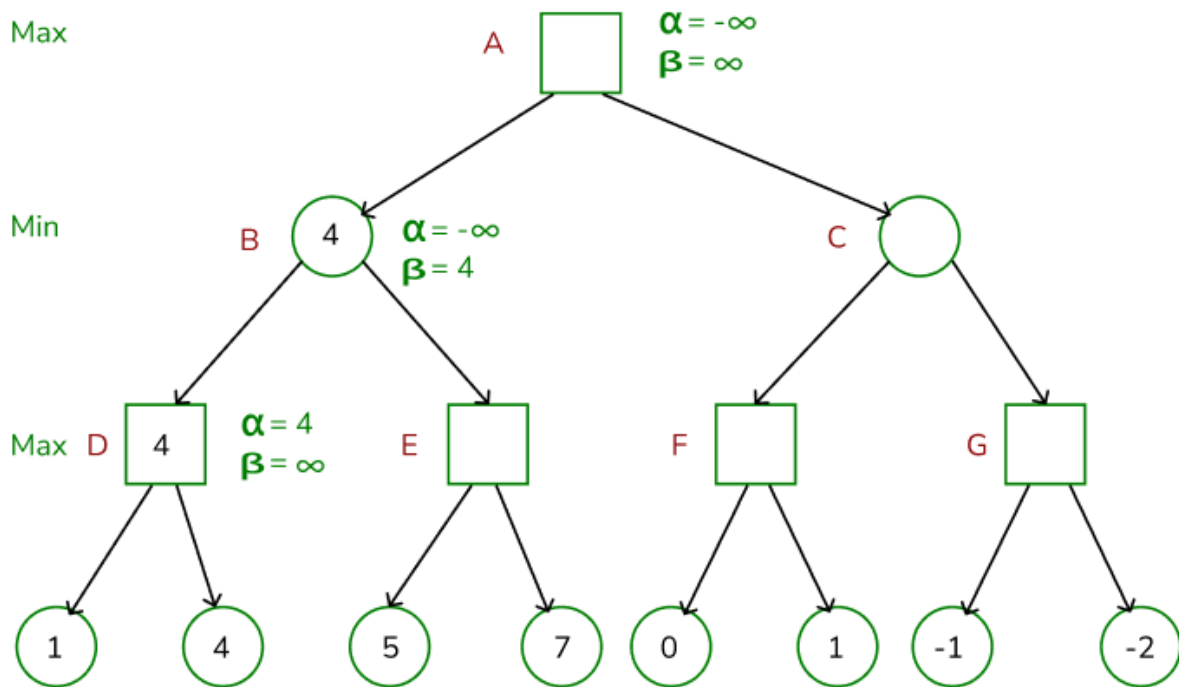
Note: alpha is the maximizer and beta is the minimizer.

Initially, the maximizer node A sets alpha to negative infinity and beta to positive infinity. Assuming that node A selects node B which inherits these values. Node B which is also a maximizer picks node D. At D, the maximizer chooses the max value (4) and then updates its alpha. Upon returning to B (minimizer), it compares its current beta with D's alpha value. If D's alpha is smaller, B's beta value gets updated. In our example, B's beta gets updated since D's alpha value is smaller.



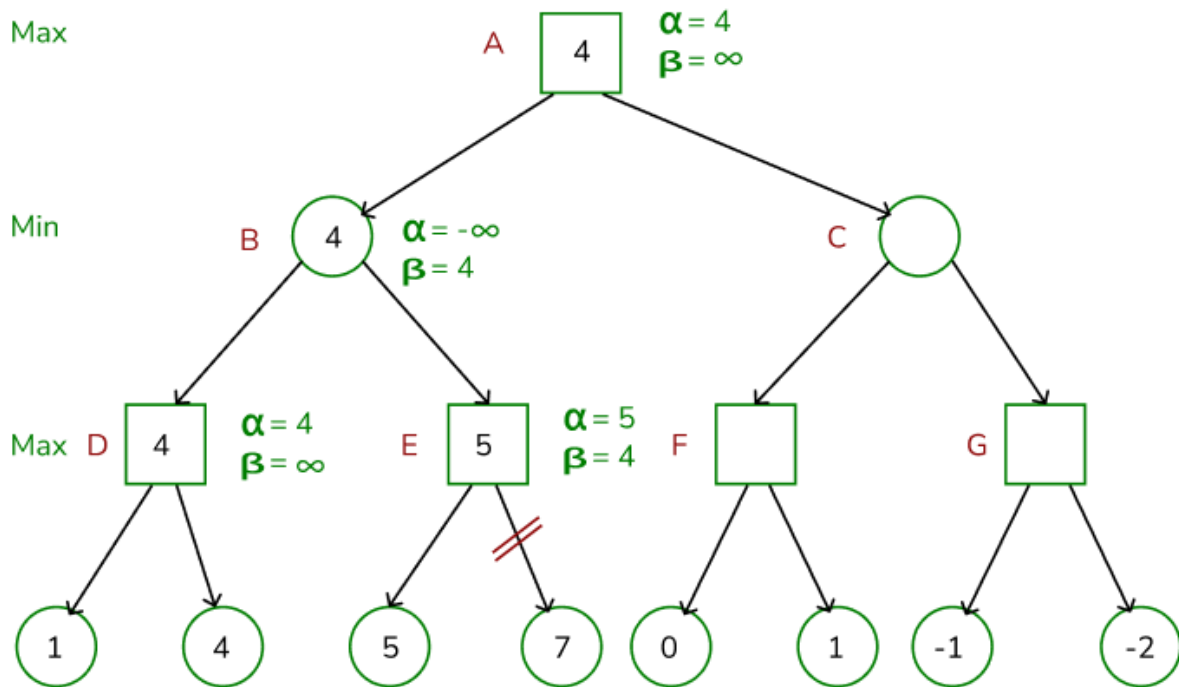
Initial game state

Now, node B selects node E and passes down its current alpha and beta values to node E. Node E inherits the alpha value of negative infinity and beta value of 4. As a maximizer, E explores its left leaf node that holds a value of 5. Since 5 is greater than negative infinity, E's alpha gets updated. Given that alpha is greater than beta, the other leaf node in E is unnecessary to explore so it gets pruned.



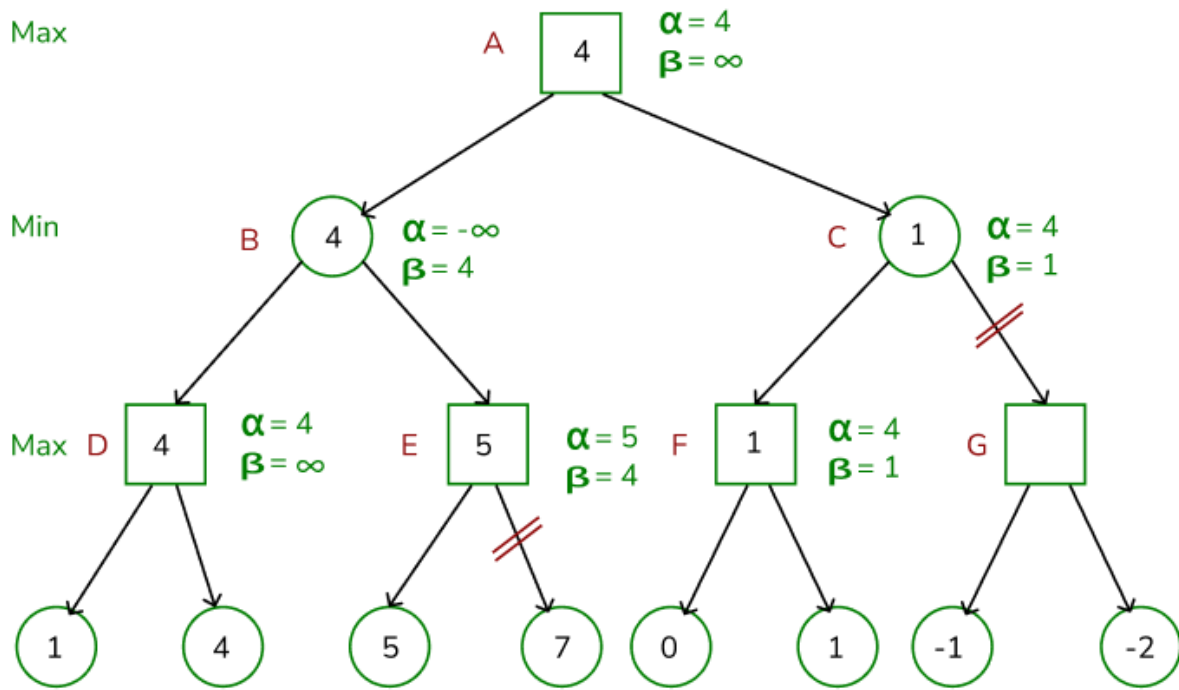
values of B and D were found

At node B, it compares its beta value with node E's alpha, if E's alpha value is lesser than B's beta updates with E's alpha. However, if E's alpha is greater than B's beta, B's beta remains unchanged. In our example, since B's beta is not greater than E's alpha, so B's beta value remains the same. Returning to node A, its alpha value updates with B's beta value because B's beta is greater than A's alpha.



The maximizer yields the value 4

At node A, the maximizer's alpha becomes 4 ensuring a value of 4 or higher. A then explores its right child which is node R. At C, alpha is 4 and beta is positive infinity. C then chooses its left child F. Node F inherits the alpha value 4 and beta value positive infinity from the parent node. F's left child yields 0 updating the F's alpha to 4 and also F's right child yields 1. Although F can achieve 1, alpha remains 4. C receives 1 from F which makes the beta 1. As alpha is greater than beta, the entire subtree of node G gets pruned. This pruning logic ensures that, as C is guaranteed 1 or less, it's irrational for A to choose C over B, where A is already guaranteed 4.



The Maximizer still holds the max value 4 and branch G gets pruned.

Adversarial search algorithm Implementations using Connect-4 Game

Connect-4 is a game played between two players who take turns dropping discs of their chosen color into a vertically suspended grid with seven columns and six rows. The objective is to be the first to form a line of four of one's discs either horizontally, vertically, or diagonally is considered a win. The game is considered solved when a player can always win or is forced to draw. This implies that there exists an optimal strategy for both players ensuring that the outcome of the game can be determined in advance. The Connect-4 game is said to be a zero-sum game because the advantage of one player will be the disadvantage of its opponent.

In this example, we will be using the Minimax algorithm to find the optimal strategy in the connect-4 game. Take a look at the below illustration, The player X chose the color red and the player O chose the color blue. In the below environment, it's now player O's turn to make a move. We will implement a minimax algorithm to find the optimal strategy for player O.

	0	1	2	3	4	5	6
0							
1							
2							
4							
4							
5							

Connect-4 game where the player O should make the optimal move to win the game

Implementation of Connect-4 using Minimax algorithm

Step 1: Check if there are any moves left

is_moves_left: This function checks if there are any empty cells left on the game board. If there are any empty cells left, it returns the value True which indicates that there are moves left to be made.

Otherwise, it returns False indicating that the game board is full and no moves can be made.

```
def is_moves_left(board):
    for row in board:
        for cell in row:
            if cell == '':
                return True
    return False
```

Step 2: Evaluate the board

evaluate: We define this function to evaluate the current state of the game board and it returns a score based on whether there is a winning configuration for a player O such as horizontally, vertically, or diagonally. If there is a winning configuration it returns a score of 10, otherwise it returns 0.

```
def evaluate(b):
    for row in range(6):
        for col in range(4):
            if b[row][col] == b[row][col+1] == b[row][col+2] == b[row][col+3] == 'o':
                return 10
    for col in range(7):
        for row in range(3):
            if b[row][col] == b[row+1][col] == b[row+2][col] == 'o':
                return 10
    for row in range(6):
        for col in range(3):
            if b[row][col] == b[row+1][col+1] == b[row+2][col+2] == 'o':
                return 10
    for row in range(6):
        for col in range(3):
            if b[row][col] == b[row+1][col-1] == b[row+2][col-2] == 'o':
                return 10
    return 0
```

```

        if b[row][col] == b[row+1][col] == b[row+2][col] == b[row+3][col] == 'o':
            return 10
    for row in range(3):
        for col in range(4):
            if b[row][col] == b[row+1][col+1] == b[row+2][col+2] == b[row+3][col+3] == 'o':
                return 10
    for row in range(3, 6):
        for col in range(3):
            if b[row][col] == b[row - 1][col+1] == b[row-2][col+2] == b[row - 3][col+3] == 'o':
                return 10
    return 0

```

Step 3: Implement the Minimax algorithm

minimax: We define the minimax function to implement the minimax algorithm that makes decision-making simpler in two-player games. It recursively evaluates the possible moves and returns the best score for the current player.

```

def minimax(board, depth, is_max):
    score = evaluate(board)

    if score == 10:
        return score - depth
    if not is_moves_left(board):
        return 0
    if is_max:
        best_value = -float('inf')
        for col in range(7):
            for row in range(5, -1, -1):
                if board[row][col] == '':
                    board[row][col] = 'x'
                    best_val = max(best_val, minimax(
                        board, depth+1, not is_max))
                    board[row][col] = ''
                    break
            return best_val
    else:
        best_value = float('inf')
        for col in range(7):
            for row in range(5, -1, -1):
                if board[row][col] == '':
                    board[row][col] = 'o'
                    best_val = max(best_val, minimax(
                        board, depth+1, not is_max))
                    board[row][col] = ''
                    break
            return best_val

```

Step 4: Find the optimal move for player O

find_optimal_move: We define this function to discover the optimal move for player O by simulating all possible moves and evaluating their scores using the minimax algorithm. It returns the coordinates of the best move.

```

def find_optimal_move(board):
    best_move = None
    best_val = -float('inf')

    for col in range(7):
        for row in range(5, -1, -1):
            if board[row][col] == '':
                board[row][col] = 'o'

```

```

        move_val = minimax(board, 0, False)
        board[row][col] = ''

        if move_val > best_val:
            best_val = move_val
            best_move = (row, col)
        break
    return best_move

```

Step 5: Test the given board configuration

Here, we will test the given board configuration and print out the optimal move for player O to win a game. If there is no possible winning move, it prints a message indicating that player O cannot win with the current board configuration.

```

board = [
    ['x', 'x', 'o', '', '', '', 'x'],
    ['o', 'o', 'o', 'x', '', '', 'x'],
    ['x', 'o', '', '', '', '', ''],
    ['x', 'o', 'o', '', '', '', ''],
    ['x', 'x', 'x', 'o', '', '', ''],
    ['o', 'o', 'x', 'o', 'x', '', '']
]
optimal_move = find_optimal_move(board)
if optimal_move:
    print("The optimal move for player O to win is:", optimal_move)
else:
    print("Player O cannot win with the current board configuration")

```

Complete code

```

def is_moves_left(board):
    for row in board:
        for cell in row:
            if cell == '':
                return True
    return False

def evaluate(b):
    for row in range(6):
        for col in range(4):
            if b[row][col] == b[row][col+1] == b[row][col+2] == b[row][col+3] == 'o':
                return 10
        for col in range(7):
            for row in range(3):
                if b[row][col] == b[row+1][col] == b[row+2][col] == b[row+3][col] ==
'o':
                    return 10
        for row in range(3):
            for col in range(4):

```

```

        if b[row][col] == b[row+1][col+1] == b[row+2][col+2] == b[row+3]
[col+3] == 'o':
            return 10
    for row in range(3, 6):
        for col in range(3):
            if b[row][col] == b[row - 1][col+1] == b[row-2][col+2] == b[row - 3]
[col+3] == 'o':
                return 10

    return 0

```

```

def minimax(board, depth, is_max):
    score = evaluate(board)

    if score == 10:
        return score - depth
    if not is_moves_left(board):
        return 0
    if is_max:
        best_value = -float('inf')
        for col in range(7):
            for row in range(5, -1, -1):
                if board[row][col] == '':
                    board[row][col] = 'x'
                    best_val = max(best_val, minimax(
                        board, depth+1, not is_max))
                    board[row][col] = ''
                    break
            return best_val
    else:
        best_value = float('inf')
        for col in range(7):
            for row in range(5, -1, -1):
                if board[row][col] == '':
                    board[row][col] = 'o'
                    best_val = max(best_val, minimax(
                        board, depth+1, not is_max))
                    board[row][col] = ''
                    break
            return best_val

```

```

def find_optimal_move(board):
    best_move = None
    best_val = -float('inf')

    for col in range(7):
        for row in range(5, -1, -1):

```

```

        if board[row][col] == '':
            board[row][col] = 'o'
            move_val = minimax(board, 0, False)
            board[row][col] = ''

            if move_val > best_val:
                best_val = move_val
                best_move = (row, col)
            break
    return best_move

```

```

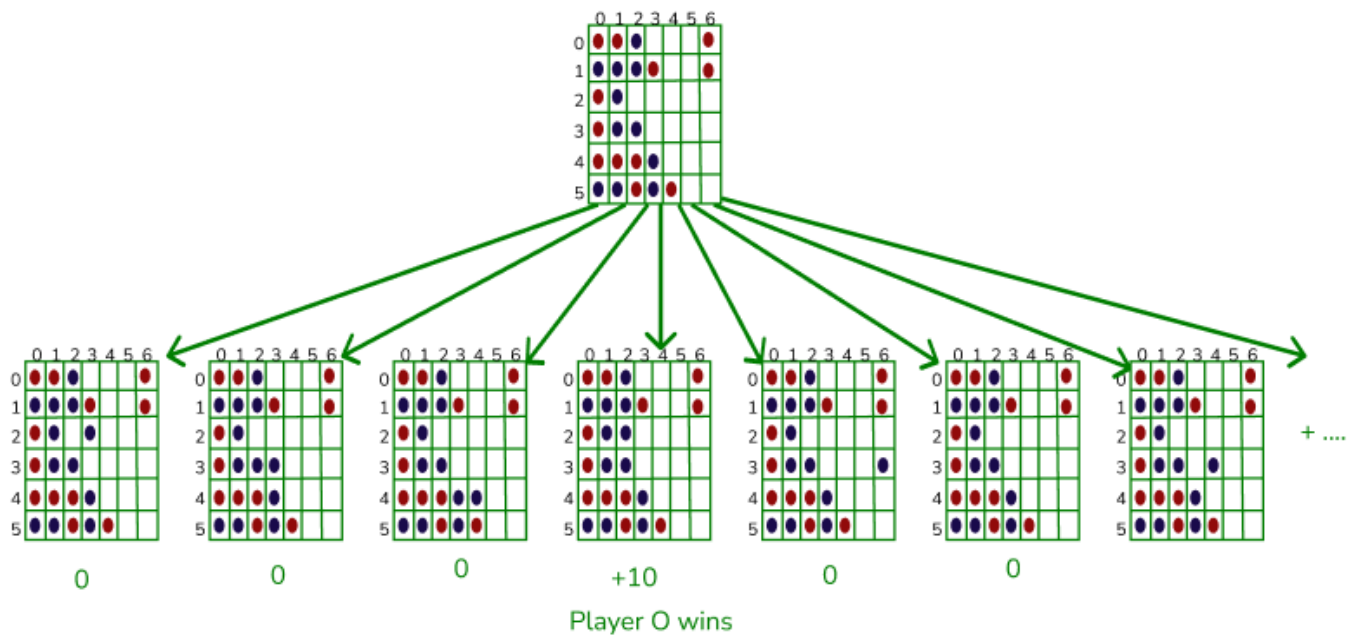
board = [
    ['x', 'x', 'o', '', '', '', 'x'],
    ['o', 'o', 'o', 'x', '', '', 'x'],
    ['x', 'o', '', '', '', '', ''],
    ['x', 'o', 'o', '', '', '', ''],
    ['x', 'x', 'x', 'o', '', '', ''],
    ['o', 'o', 'x', 'o', 'x', '', '']
]
optimal_move = find_optimal_move(board)
if optimal_move:
    print("The optimal move for player 0 to win is:", optimal_move)
else:
    print("Player 0 cannot win with the current board configuration")

```

Output:

The optimal move for player 0 to win is: (2,2)

Output Explanation



Output of Connect-4 where the player O wins

Take a look at the above output illustration, the player O won when he chose the optimal move such as (2,2). It's important to note that there are multiple possible moves that player O could have made to potentially win the game. However, in the given illustration only seven states of the game board are shown to demonstrate the process of finding the optimal move using the minimax algorithm.

Applications of adversarial search algorithms

- **Board games:** Adversarial search is most widely used in various board games like Chess, Checkers, Go and Connect Four. The above-explained algorithms can help the computers to play against human opponents or other computer players.
- **Game Theory:** Adversarial search forms the basis of [game theory](#), which is used in various fields like economics, political science, and biology to model strategic interactions between rational decision-makers.
- **Puzzle-solving:** Adversarial search algorithms can be used to solve puzzles and optimization problems where the goal is to find the best sequence of moves or actions to achieve a desired outcome.

Conclusion

Adversarial search algorithms have emerged as a powerful tool with diverse applications across numerous domains. From mastering complex board games to enhancing cybersecurity, robotics, and automated negotiation systems, these algorithms facilitate strategic decision-making in dynamic competitive environments.

Comment

T tejashr...

5

Article Tags:

Blogathon Artificial Intelligence AI-ML-DS

Explore

Introduction to AI	▼
AI Concepts	▼
Machine Learning in AI	▼
Robotics and AI	▼
Generative AI	▼
AI Practice	▼



Corporate & Communications Address:
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

Registered Address:
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305



Company	Explore	Tutorials	Courses	Videos
About Us	POTD	Programming	ML and Data	DSA
Legal	Job-A-Thon	Languages	Science	Python
Privacy	Blogs	DSA	DSA and	Java
Policy	Nation	Web	Placements	C++
Contact Us	Skill Up	Technology	Web	Web
Advertise with us		AI, ML &	Development	Developer
GFG		Data Science	Programming	Data Science
Corporate Solution		DevOps	Languages	CS Subjects
Training Program		CS Core Subjects	DevOps & Cloud	
		Interview Preparation	GATE	
		Software and Tools	Trending Technologies	